TSwap Protocol Audit Report

Audit Details

- Commit Hash: <insert commit hash — run `git log -1 --format=%H` in your `t-swap-audit` repo root>

Scope

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

Roles

- **Liquidity Provider (LP)** — Deposits WETH and a paired pool token into a `TSwapPool` via `deposit`, receiving liquidity tokens (an ERC20 minted by the pool itself) representing their proportional share of the pool. Can withdraw their share (plus accrued fees) via `withdraw`.
- **Swapper/Trader** — Exchanges WETH for the pool token (or vice versa) via `swapExactInput` or `swapExactOutput`, paying a 0.3% fee that accrues to the pool's reserves for the benefit of LPs.
- **Pool Deployer** — Creates a new `TSwapPool` instance for a given ERC20/WETH pair via `PoolFactory::createPool`. No special privileges over an existing pool once deployed; `TSwapPool` itself has no owner/admin role.

Executive Summary

This audit was conducted on the TSwap protocol — a constant-product automated market maker (AMM) enabling liquidity provision and token swaps between WETH and a paired ERC20 token, structurally modeled on Uniswap V2.

The review combined manual line-by-line analysis, static analysis tooling, and extensive stateful fuzz/invariant testing against the protocol's core `x * y = k` pricing invariant. The invariant testing itself proved highly effective: it programmatically surfaced a High severity finding (the swap-count reward mechanism draining reserves outside the pricing formula) that would have been difficult to catch through manual review alone, demonstrating real value from combining both approaches.

The audit identified four High severity findings — a scaling error in the input-pricing formula causing systematic overcharging, a missing deadline check on deposits, the absence of slippage protection on `swapExactOutput`, and an input/output parameter mismatch in `sellPoolTokens` that inverts the function's intended behavior — plus additional Medium severity findings concerning non-standard token compatibility (fee-on-transfer, rebasing, and ERC777 tokens) and other issues documented below.

Collectively, these findings indicate the core AMM math is sound under standard usage (validated via 1000-run invariant campaigns with zero violations once test tooling itself was debugged), but several surrounding functions — pricing formulas, parameter validation, and token-standard assumptions — contain concrete, exploitable errors that should be remediated before mainnet deployment.

Issues Found

Severity	Number of Issues Found
High	4
Medium	2
Low	—
Informational	—
Gas	—
Total	**6**

(Update the Low/Informational/Gas counts and this table once those sections are finalized — see placeholders below.)

Findings

High

[H-1] `TSwapPool::deposit` is missing deadline validation, causing transactions to execute even after their deadline has passed

(Full finding as drafted — the `deadline` parameter is accepted but never checked via `revertIfDeadlinePassed`, unlike every other state-changing function in the contract.)

[H-2] `TSwapPool::getInputAmountBasedOnOutput` uses an incorrect scaling constant, causing users to be overcharged on every `swapExactOutput` and `sellPoolTokens` call

(Full finding as drafted — uses `10000` instead of `1000` as the fee-scaling denominator, causing a confirmed ~10x overcharge, backed by PoC test `test_getInputAmountBasedOnOutputOverchargesUser`.)

[H-3] `TSwapPool::swapExactOutput` has no slippage protection, allowing users to be charged an arbitrarily high input amount

(Full finding as drafted — no `maximumInputAmount` parameter exists, unlike `swapExactInput`'s `minOutputAmount`, backed by PoC tests `test_swapExactOutput_noSlippageProtection` and `test_swapExactOutput_hasNoMaximumInputParameter`.)

[H-4] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

(Full finding as drafted — calls `swapExactOutput` instead of `swapExactInput`, inverting the function's intended sell-side semantics, backed by PoC test `test_sellPoolTokensMismatchesInputAndOutput`.)

Medium

[M-1] *(reserved — insert here if you have an existing M-1 from earlier work not captured in this conversation; renumber M-2 below to M-1 if not)*

[M-2] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

(Full finding as drafted — the pool assumes `balanceOf`/`transferFrom` behave like standard ERC20, breaking for fee-charging, rebasing, or hook-based tokens, backed by PoC test `test_feeOnTransferTokenBreaksInvariant`.)

Low

(Insert any Low severity findings here — e.g., the `getActivePlayerIndex`-style ambiguous return value pattern if applicable to TSwap, or any minor issues surfaced by Slither/Aderyn that don't rise to

Medium/High. None have been finalized for TSwap yet in this conversation — populate as you identify them.)

Informational

(Insert Informational findings here — e.g., floating pragma checks, NatSpec completeness, event indexing on `LiquidityAdded`/`LiquidityRemoved`/`Swap`/`FeeAddressChanged`-equivalent events if TSwap has any unindexed address parameters, timestamp-dependence notes, etc. Populate from your Slither output once `slither .` finishes running successfully.)

Gas

(Insert Gas optimization findings here — e.g., caching storage reads like `i_poolToken.balanceOf(address(this))` when read multiple times within `deposit`, the unused `poolTokenReserves` local variable flagged by the compiler warning in `deposit`, or `constant`/`immutable` opportunities. Populate from your Slither/Aderyn output once available.)

High

[H-1] The swap-count reward mechanism in `TSwapPool::_swap` gives away free tokens outside the constant-product formula, breaking the core `x * y = k` invariant and draining pool reserves

Description: Every 10th call to `_swap` awards the caller an extra 1e18 tokens as a loyalty incentive:

```
1      swap_count++;
2      if (swap_count >= SWAP_COUNT_MAX) {
3          swap_count = 0;
4          outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000)
               ;
5      }
```

This transfer happens completely outside of, and unaccounted for by, the constant-product formula (`getOutputAmountBasedOnInput`/`getInputAmountBasedOnOutput`). The pool's reserves decrease by more than the swap's actual priced output, silently breaking the `weth * poolTokens = k` invariant the entire protocol depends on fair, manipulation-resistant pricing.

Impact: Every 10th swap permanently drains 1 full token (1e18) from the pool's reserves, with no corresponding payment. This value comes directly out of LP's pooled assets. An attacker can trigger

this deliberately and predictably - perfoming 9 minimal swaps to advance the conter, then executing the 10th swap themselves to collect the free token - repeatedly draining the pool over time.

Proof of Concept: Demonstrated via invariant fuzz testing. A 10-call sequence of `swapPoolTokenForWethBased` was found (via `forge test --mt invariant_constantProductFormulaStaysTheSameY -vvvv`) where the actual WETH reserve change (`-1000000001000000003`) diverged from the formula-predicted change (`-1000000003`) by exactly `1000000000000000000` (1e18) - precisely matching the hardcoded bonus payout, on the 10th swap in the sequence.

Run:

```
forge test --mt invariant_constantProductFormulaStaysTheSameY -vvvv
```

Recommended Mitigation: Remove the swap-count reward mechanism entirely, since it structurally conflicts with the protocol's core pricing invariant. If a loyalty/incentive mechanism is desired, fund it from a separate, dedicated rewards pool - never from the swap reserves that back the constant-product formula.

[H-2] `TSwapPool::getInputAmountBasedOnOutput` uses an incorrect scaling constant, causing users to be overcharged on every `swapExactOutput` and `sellPoolTokens` call

Description: `getInputAmountBasedOnOutput` calculates how much input a user must supply to receive a desired output amount, applying the protocol's 0.3% swap fee in the process:

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
    return
        ((inputReserves * outputAmount) * 10000) /
@>      ((outputReserves - outputAmount) * 997);
}
```

Compare this to the sibling pricing function, `getOutputAmountBasedOnInput`, which correctly applies the same 0.3% fee using a `997/1000` scaling factor:

```
function getOutputAmountBasedOnInput(
    uint256 inputAmount,
    uint256 inputReserves,
```

```
    uint256 outputReserves
)
    public
    pure
    ...
{
    uint256 inputAmountMinusFee = inputAmount * 997;
    uint256 numerator = inputAmountMinusFee * outputReserves;
    uint256 denominator = (inputReserves * 1000) + inputAmountMinusFee;
    return numerator / denominator;
}
```

`getInputAmountBasedOnOutput` uses `10000` instead of the correct `1000` as its scaling denominator. This is not a stylistic inconsistency — it's a mathematical error that changes the effective fee charged from the intended 0.3% to a dramatically higher, incorrect rate.

Impact: Every user who calls `swapExactOutput` (and, by extension, `sellPoolTokens`, which wraps `swapExactOutput`) is charged significantly more input tokens than the protocol's advertised 0.3% fee actually requires. Instead of paying `input * 1000/997` (approx. 0.3009% effective fee) to receive their desired output, users are forced to pay `input * 10000/997` — roughly **10x** the correct amount relative to the intended fee structure.

This is a direct, silent overcharge on every single `swapExactOutput` transaction. Depending on the specific reserve ratios involved, this can result in users paying dramatically more than expected for a given output, and represents a significant and consistent value leak away from users and toward whichever side of the pool benefits from the miscalculated ratio.

Proof of Concept:

Consider a pool with `inputReserves = 100e18` and `outputReserves = 100e18`, and a user wants `outputAmount = 1e18`:

- **Correct formula (mirroring `getOutputAmountBasedOnInput`'s fee logic, using `1000` instead of `10000`):** `inputAmount = (100e18 * 1e18 * 1000)/ ((100e18 - 1e18)* 997)approx. 1.0107e18`
- **Actual (buggy) formula, as currently implemented:** `inputAmount = (100e18 * 1e18 * 10000)/ ((100e18 - 1e18)* 997)approx. 10.107e18`

The buggy version demands roughly **10x** the input tokens the correctly-scaled fee formula would require for the same output.

PoC

```
function test_getInputAmountBasedOnOutputOverchargesUser() public view
    {
    uint256 outputAmount = 1e18;
```

```
    uint256 inputReserves = 100e18;
    uint256 outputReserves = 100e18;

    uint256 actualInput = pool.getInputAmountBasedOnOutput(outputAmount
        , inputReserves, outputReserves);

    // What the input SHOULD be if the fee were correctly scaled by
        1000, not 10000
    uint256 correctInput = ((inputReserves * outputAmount) * 1000) / ((
        outputReserves - outputAmount) * 997);

    console2.log("Actual (buggy) input required: ", actualInput);
    console2.log("Correct input required: ", correctInput);

    assertGt(actualInput, correctInput * 9); // confirms roughly 10x
        overcharge
}
```

Run:

```
forge test --mt test_getInputAmountBasedOnOutputOverchargesUser -vv
```

Recommended Mitigation: Correct the scaling constant to `1000`, matching the fee logic used in `getOutputAmountBasedOnInput`:

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
-           ((inputReserves * outputAmount) * 10000) /
+           ((inputReserves * outputAmount) * 1000) /
            ((outputReserves - outputAmount) * 997);
    }
```

[H-3] `TSwapPool::swapExactOutput` has no slippage protection, allowing users to be charged an arbitrarily high input amount

Description: `swapExactOutput` lets a caller specify exactly how much output they want, then calculates and charges whatever `inputAmount` the pricing formula determines is required:

```
function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
    uint256 outputAmount,
    uint64 deadline
)
    public
    revertIfZero(outputAmount)
    revertIfDeadlinePassed(deadline)
    returns (uint256 inputAmount)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

    inputAmount = getInputAmountBasedOnOutput(
        outputAmount,
        inputReserves,
        outputReserves
    );

@>  _swap(inputToken, inputAmount, outputToken, outputAmount);
}
```

Unlike `swapExactInput`, which requires a `minOutputAmount` parameter and reverts if the actual output would be less than the caller expects:

```
function swapExactInput(
    IERC20 inputToken,
    uint256 inputAmount,
    IERC20 outputToken,
    uint256 minOutputAmount,
    uint64 deadline
)
    ...
{
    ...
    if (outputAmount < minOutputAmount) {
        revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
    }
    ...
}
```

`swapExactOutput` has **no equivalent protection on the input side**. There is no `maximumInputAmount` parameter, and no check comparing the computed `inputAmount` against any caller-supplied ceiling. Whatever `getInputAmountBasedOnOutput` returns, the caller is charged — no matter how unfavorable.

Impact: Between the moment a user signs a `swapExactOutput` transaction and the moment it's actually mined, the pool's reserves can shift significantly due to other swaps executing first (organi-

cally, or via deliberate front-running/MEV). Since there is no cap on `inputAmount`, the user has no on-chain guarantee about the maximum price they'll pay for their desired output.

A malicious actor (or MEV bot) can observe a pending `swapExactOutput` transaction in the mempool and front-run it with trades that skew the pool's reserve ratio, causing `getInputAmountBasedOnOutput` to compute a dramatically higher `inputAmount` than the user expected when they signed the transaction — then back-run it to restore the ratio and capture the difference (a classic sandwich attack). The user's transaction will still succeed, silently charging them far more than intended, since nothing in the function reverts on an unfavorable price.

This is compounded by the separate `getInputAmountBasedOnOutput` scaling bug (H-#), since users are already being overcharged by the formula itself — with no slippage cap in place, that overcharge has no upper bound at all under adverse market/MEV conditions.

Proof of Concept:

1. User A calls `swapExactOutput` intending to receive `1000` output tokens, expecting to pay approximately `X` input tokens based on the pool's current reserves.
2. Before User A's transaction is mined, an attacker submits a transaction that swaps heavily in a direction that shifts the reserve ratio unfavorably for User A.
3. User A's transaction is mined after the attacker's, and `getInputAmountBasedOnOutput` now computes a much larger `inputAmount` than User A anticipated.
4. Because there is no `maximumInputAmount` check, the swap proceeds anyway, charging User A significantly more than they were willing to pay — with no way to have prevented this on-chain.

Recommended Mitigation: Add a `maximumInputAmount` parameter, mirroring the protection `swapExactInput` already provides via `minOutputAmount`:

```
+   error TSwapPool__InputTooHigh(uint256 actual, uint256 max);
    function swapExactOutput(
        IERC20 inputToken,
        IERC20 outputToken,
        uint256 outputAmount,
+       uint256 maximumInputAmount,
        uint64 deadline
    )
        public
        revertIfZero(outputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 inputAmount)
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
        uint256 outputReserves = outputToken.balanceOf(address(this));

        inputAmount = getInputAmountBasedOnOutput(
```

```
18              outputAmount,
19              inputReserves,
20              outputReserves
21          );
22
23  +       if (inputAmount > maximumInputAmount) {
24  +           revert TSwapPool__InputTooHigh(inputAmount,
        maximumInputAmount);
25  +       }
26
27          _swap(inputToken, inputAmount, outputToken, outputAmount);
28      }
```

This ensures the transaction reverts rather than silently executing at an unfavorable price if reserves shift beyond the caller's tolerance between signing and mining.

[H-4] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to let users easily sell pool tokens in exchange for WETH, with `poolTokenAmount` representing the exact amount of pool tokens the user wants to sell:

```
1  function sellPoolTokens(uint256 poolTokenAmount) external returns (
       uint256 wethAmount) {
2      return swapExactOutput(
3          i_poolToken,
4          i_wethToken,
5  @>      poolTokenAmount,
6          uint64(block.timestamp)
7      );
8  }
```

However, `poolTokenAmount` is passed as the `outputAmount` argument to `swapExactOutput`, not as an input amount:

```
1  function swapExactOutput(
2      IERC20 inputToken,
3      IERC20 outputToken,
4      uint256 outputAmount,
5      uint64 deadline
6  ) public ... returns (uint256 inputAmount) { ... }
```

Since `swapExactOutput`'s semantics are "give me exactly `outputAmount` of `outputToken`, and calculate however much `inputToken` that costs," calling it this way tells the pool **"give the user exactly `poolTokenAmount` WETH,"** not "sell `poolTokenAmount` pool tokens." This is the wrong

function entirely for what `sellPoolTokens` is meant to do — `swapExactInput` is the correct function to call here, since the user is specifying an exact input amount (pool tokens they're selling), not an exact output amount.

Impact: Users calling `sellPoolTokens(poolTokenAmount)` expecting to sell exactly `poolTokenAmount` of their pool tokens are instead requesting exactly `poolTokenAmount` **WETH as output** — a completely different, and typically much larger, value depending on the pool's exchange rate. This can result in:

- The user being charged a wildly different (often far greater) amount of pool tokens than they intended to sell, since the required input is now derived from wanting `poolTokenAmount` WETH out, not from selling `poolTokenAmount` pool tokens in.
- Unexpected reverts if the calculated required input exceeds the user's token balance or allowance.
- Silent, severe financial loss for any user who assumes `sellPoolTokens(100)` means "sell 100 pool tokens," when it actually means "acquire 100 WETH, paying however many pool tokens that requires."

Proof of Concept:

PoC

```
function test_sellPoolTokensMismatchesInputAndOutput() public {
    // Set up the pool with initial liquidity: 100 WETH / 100 poolToken
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    // User intends to sell exactly 10 poolTokens for WETH
    uint256 poolTokensUserWantsToSell = 10e18;

    // Give user enough poolToken to have a safety margin, so the
       mismatch doesn't just revert
    // on an allowance/balance issue and instead demonstrates the
       ACTUAL miscalculation
    poolToken.mint(user, 1_000e18);

    vm.startPrank(user);
    poolToken.approve(address(pool), 1_000e18);

    uint256 userPoolTokenBalanceBefore = poolToken.balanceOf(user);
    uint256 userWethBalanceBefore = weth.balanceOf(user);

    // User calls sellPoolTokens expecting to sell ~10 poolTokens
    uint256 wethReceived = pool.sellPoolTokens(
```

```
            poolTokensUserWantsToSell);

        uint256 userPoolTokenBalanceAfter = poolToken.balanceOf(user);
        uint256 poolTokensActuallySpent = userPoolTokenBalanceBefore -
            userPoolTokenBalanceAfter;

        console2.log("PoolTokens user intended to sell: ",
            poolTokensUserWantsToSell);
        console2.log("PoolTokens user ACTUALLY spent:    ",
            poolTokensActuallySpent);
        console2.log("WETH received (requested as EXACT OUTPUT, not derived
             from selling 10 poolTokens): ", wethReceived);

        // The user received exactly `poolTokensUserWantsToSell` WETH (10
            e18) as output,
        // NOT the WETH equivalent of selling 10 poolTokens as input —
            proving the mismatch
        assertEq(wethReceived, poolTokensUserWantsToSell);

        // The poolTokens actually spent to acquire that WETH will differ
            significantly from
        // the 10 poolTokens the user intended to sell, since the pricing
            direction is inverted
        assertTrue(poolTokensActuallySpent != poolTokensUserWantsToSell);
    }
```

Run:

```
forge test --mt test_sellPoolTokensMismatchesInputAndOutput -vv
```

The test confirms `sellPoolTokens(10e18)` results in the user receiving **exactly 10 WETH** (an output amount), rather than the WETH proceeds from selling 10 pool tokens (an input amount) — demonstrating that the function's actual behavior is the inverse of its intended and documented purpose.

Recommended Mitigation: Call `swapExactInput` instead of `swapExactOutput`, since `sellPoolTokens` is meant to accept an exact input amount from the user:

```
    function sellPoolTokens(
        uint256 poolTokenAmount
    ) external returns (uint256 wethAmount) {
-       return swapExactOutput(
-           i_poolToken,
-           i_wethToken,
-           poolTokenAmount,
-           uint64(block.timestamp)
-       );
+       return swapExactInput(
+           i_poolToken,
+           poolTokenAmount,
```

```
+            i_wethToken,
+            minWethToReceive,
+            uint64(block.timestamp)
+        );
     }
```

Note this also requires adding a `minWethToReceive` parameter to `sellPoolTokens` itself, since `swapExactInput` requires a `minOutputAmount` for slippage protection:

```
     function sellPoolTokens(
-        uint256 poolTokenAmount
+        uint256 poolTokenAmount,
+        uint256 minWethToReceive
     ) external returns (uint256 wethAmount) {
         return swapExactInput(
             i_poolToken,
             poolTokenAmount,
             i_wethToken,
             minWethToReceive,
             uint64(block.timestamp)
         );
     }
```

This both fixes the input/output mismatch and gives callers of `sellPoolTokens` the same slippage protection already available through `swapExactInput` directly.

Mediums

[M-1] `TSwapPool::deposit` is missing deadline validation, causing transactions to execute even after their deadline has passed

Description: The `deposit` function accepts a `deadline` parameter, but it is never actually checked anywhere inside the function body:

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
@>  uint64 deadline
)
    external
    revertIfZero(wethToDeposit)
    returns (uint256 liquidityTokensToMint)
{
    if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
        revert TSwapPool__WethDepositAmountTooLow(
            MINIMUM_WETH_LIQUIDITY,
```

```
            wethToDeposit
        );
    }
    // ... rest of function never references `deadline`
```

Every other state-changing function in the codebase that accepts a `deadline` parameter (`swapExactInput`, `swapExactOutput`, `withdraw`) applies the `revertIfDeadlinePassed(deadline)` modifier to enforce it:

```
modifier revertIfDeadlinePassed(uint64 deadline) {
    if (deadline < uint64(block.timestamp)) {
        revert TSwapPool__DeadlineHasPassed(deadline);
    }
    _;
}
```

`deposit` is conspicuously missing this modifier, despite accepting the same `deadline` parameter and clearly being intended to support the same protection.

Impact: A user submitting a `deposit` transaction with a specific deadline in mind (e.g., "only execute this within the next block or two, otherwise the market may have moved") has no actual guarantee that their transaction will be rejected after that point. If the transaction sits in the mempool and is mined much later — whether due to network congestion, a miner/validator deliberately delaying it, or simple bad luck — it will still execute using whatever pool reserves exist at that later time, even though the user's intended deadline has long since passed.

This exposes depositors to unexpected slippage and MEV-style manipulation: a malicious actor (or miner) can hold a pending deposit transaction and strategically time its inclusion to a block where the pool's reserve ratio is less favorable to the depositor, since there is no on-chain mechanism forcing the deposit to be rejected once the user's intended time window has elapsed. While `minimumLiquidityTokensToMint` and `maximumPoolTokensToDeposit` do offer some slippage protection, they don't address the core issue that the transaction can be executed at an arbitrary, unbounded point in the future.

Proof of Concept:

1. A user calls `deposit(wethAmount, minLpTokens, maxPoolTokens, deadline)`, setting `deadline` to a timestamp representing "the next block or two."
2. Due to network conditions (or deliberate action by a block producer), the transaction is not mined until significantly later — well past `deadline`.
3. The transaction still executes successfully, since `deadline` is never checked, potentially depositing at a much less favorable pool ratio than the user intended when they signed the transaction.

Recommended Mitigation: Apply the existing `revertIfDeadlinePassed` modifier to `deposit`, consistent with every other state-changing function in the contract:

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
        revertIfZero(wethToDeposit)
+       revertIfDeadlinePassed(deadline)
        returns (uint256 liquidityTokensToMint)
    {
```

[M-2] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

Description: `TSwapPool` assumes that the amount of tokens it requests via `transferFrom` is exactly the amount it actually receives, and that its recorded balances always match the token's own `balanceOf(address(this))` over time. This assumption does not hold for several common non-standard ERC20 token designs:

- **Fee-on-transfer / deflationary tokens** — skim a percentage on every transfer, so the pool receives *less* than the amount requested (as we demonstrated earlier in a different protocol's fee-on-transfer accounting bug).
- **Rebase tokens** — periodically and automatically adjust every holder's balance (e.g., to reflect staking yield or supply contraction), meaning the pool's balance of the token can change **without any transfer occurring at all**.
- **ERC777 tokens** — trigger hooks (`tokensReceived`/`tokensToSend`) on transfer, opening a reentrancy surface distinct from ERC20, and can also implement custom transfer logic that doesn't guarantee amount-for-amount delivery.

`enterRaffle`-style accounting is not present here, but the same root issue applies throughout `TSwapPool`: every function that relies on `i_wethToken.balanceOf(address(this))` or `i_poolToken.balanceOf(address(this))` immediately before/after a transfer — `deposit`, `withdraw`, `_swap`, and the pricing functions that read reserves — implicitly assumes those balances behave like a standard, non-rebasing, non-fee-charging ERC20.

Impact: If a pool is created for a fee-on-transfer, rebasing, or ERC777-style token:

- **Fee-on-transfer:** Deposits and swaps will record more tokens as "received" than the pool actually holds, corrupting the `x * y = k` invariant and causing the internal accounting to

overstate real reserves — mirroring the exact vulnerability class demonstrated earlier in the `HandlerStatefulFuzzCatches` fee-on-transfer PoC.

- **Rebase tokens:** The pool's actual token balance can silently drift out of sync with what the constant-product formula assumes, since a rebase event changes `balanceOf(address(this))` without any `deposit`/`withdraw`/`swap` call ever happening. This can permanently desynchronize the pool's pricing from its real backing.
- **ERC777:** The transfer hooks introduce a reentrancy vector that the current `_swap`/`_addLiquidityMintAndTransfer` CEI-pattern ordering does not account for, since ERC777 hooks execute mid-transfer, not merely after — potentially allowing state manipulation between the pool's internal calculations and the actual token movement.

In all three cases, the core invariant this protocol depends on for fair pricing can be broken, potentially locking funds, mispricing swaps, or enabling further exploitation depending on the specific non-standard token's behavior.

Proof of Concept:

PoC (fee-on-transfer scenario)

```
contract FeeOnTransferToken is ERC20Mock {
    uint256 public constant FEE_BPS = 100; // 1% fee on every transfer

    function transfer(address to, uint256 amount) public override
        returns (bool) {
        uint256 fee = (amount * FEE_BPS) / 10000;
        uint256 amountAfterFee = amount - fee;
        return super.transfer(to, amountAfterFee);
    }

    function transferFrom(address from, address to, uint256 amount)
        public override returns (bool) {
        uint256 fee = (amount * FEE_BPS) / 10000;
        uint256 amountAfterFee = amount - fee;
        return super.transferFrom(from, to, amountAfterFee);
    }
}

function test_feeOnTransferTokenBreaksInvariant() public {
    FeeOnTransferToken feeToken = new FeeOnTransferToken();
    TSwapPool feeTokenPool = new TSwapPool(address(feeToken), address(
        weth), "LFeeToken", "LFT");

    feeToken.mint(liquidityProvider, 100e18);
    weth.mint(liquidityProvider, 100e18);

    vm.startPrank(liquidityProvider);
    feeToken.approve(address(feeTokenPool), 100e18);
    weth.approve(address(feeTokenPool), 100e18);
```

```
27
28      // Deposit requests 100e18 feeToken, but only ~99e18 actually
            arrives due to the 1% fee
29      feeTokenPool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp
            ));
30      vm.stopPrank();
31
32      uint256 actualFeeTokenBalance = feeToken.balanceOf(address(
            feeTokenPool));
33      console2.log("Expected pool feeToken balance: ", 100e18);
34      console2.log("Actual pool feeToken balance:   ",
            actualFeeTokenBalance);
35
36      // Confirms the pool's actual holdings are less than what deposit()
             assumed it received
37      assertLt(actualFeeTokenBalance, 100e18);
38  }
```

Run:

```
1  forge test --mt test_feeOnTransferTokenBreaksInvariant -vv
```

This confirms the pool's actual token balance falls short of what its internal accounting assumes, immediately corrupting the constant-product ratio the protocol depends on.

Recommended Mitigation:

1. **Document and enforce a standard-ERC20-only policy.** The simplest mitigation is to explicitly disallow non-standard tokens at the protocol level — clearly document that `TSwapPool` is only safe to deploy for standard, non-fee-charging, non-rebasing ERC20 tokens, and consider adding a factory-level allowlist or a deployment-time check (e.g., verifying a test transfer moves the exact expected amount) to reject incompatible tokens before a pool is created.
2. **If support for fee-on-transfer tokens is required**, measure actual balance deltas instead of trusting the requested transfer amount:

```
1  +    uint256 balanceBefore = i_poolToken.balanceOf(address(this));
2       i_poolToken.safeTransferFrom(msg.sender, address(this),
            poolTokensToDeposit);
3  +    uint256 actualAmountReceived = i_poolToken.balanceOf(address(this))
        - balanceBefore;
```

and use `actualAmountReceived` in place of the requested amount for all subsequent accounting.

3. **For ERC777 compatibility specifically**, add reentrancy guards (e.g., OpenZeppelin's `ReentrancyGuard`) to all state-changing functions that move tokens, since the CEI pattern alone does not fully protect against hooks that execute mid-transfer.

4. **Rebase tokens are fundamentally incompatible** with the constant-product model as implemented and should be explicitly excluded rather than patched around, since balance changes can occur with no corresponding function call for the pool to react to.

Lows

[L-1] `TSwapPool::LiquidityAdded` event has parameters out of order causing event to emit incorrect information

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityMintAndTran` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

```
1 -     emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit
  );
2 +     emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit
  );
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Recommended Mitigation:

```
1         uint256 inputReserves = inputToken.balanceOf(address(this));
2         uint256 outputReserves = outputToken.balanceOf(address(this));
3
4 -         uint256 outputAmount = getOutputAmountBasedOnInput(
              inputAmount, inputReserves, outputReserves);
5 +         uint256 output = getOutputAmountBasedOnInput(
  inputAmount, inputReserves, outputReserves);
6
7 -         if (outputAmount < minOutputAmount) {
8 -             revert TSwapPool__OutputTooLow(outputAmount,
  minOutputAmount);
9 -         }
```

```
10 +       if (output < minOutputAmount) {
11 +           revert TSwapPool__OutputTooLow(outputAmount,
       minOutputAmount);
12 +       }
13
14 -       _swap(inputToken, inputAmount, outputToken, outputAmount);
15 +       _swap(inputToken, inputAmount, outputToken, output);
```

Informationals

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed

```
1 -   error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking zero address checks

```
1      constructor(address wethToken) {
2 +        if(wethToken == address(0)){
3 +            revert();
4 +        }
5      }
```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`

```
1 -    string memory liquidityTokenSymbol = string.concat("ts", IERC20(
       tokenAddress).name());
2 +    string memory liquidityTokenSymbol = string.concat("ts", IERC20(
       tokenAddress).symbol());
```